



Algoritmos y Programación II
Trabajo Práctico 2
2° Cuatrimestre de 2021

Alumnos: Luciano Bogado de Oliveira Manarte - 107724 - lbogado@fi.uba.ar

Manuel Lo Coco - 103938 - mlococo@fi.uba.ar

Corrector: Jasmina Sella Faena

INICIO DE LA EJECUCIÓN:

Al iniciar la ejecución del programa se lee un documento de texto que es ingresado por entrada estándar. Para leer este documento se usó la función `getline` la cual lee el archivo línea a línea. Este documento contiene los usuarios de la aplicación que desarrollamos estos usuarios son guardados en dos tablas de hash.

Las tablas de hash se utilizaron para poder acceder a los datos guardados en ellas en complejidad $O(1)$. Ambas estructuras de datos usaron como claves los usuarios que se leen a través del documento de texto.

En el primer hash se guardó la posición de los usuarios en el documento de texto. La posición fue guardada para poder sacar la distancia entre dos usuarios en el documento de texto. Para sacar esta distancia lo que se hizo fue restar las dos posiciones y calcular su módulo. Estas distancias se calculan en el comando publicar un post ya que estas distancias son la afinidad del usuario con el publicador del post.

En el segundo hash se guardó un heap vacío para cada usuario el cual más tarde va a ser el feed de los usuarios donde se van a guardar los post, por prioridad según la afinidad con el usuario.

COMANDOS IMPLEMENTADOS:

1_ LOGIN:

Para el login se utilizó uno de los hashes que se crearon al inicio. De esta forma se buscaba el usuario ingresado por la entrada estándar en el hash. El usuario se guarda en una variable `char*`. Se utilizó un hash, porque la primitiva hash pertenece, busca si hay una clave en el hash en complejidad $O(1)$ la cual era la complejidad pedida para este comando. Si el usuario no se encuentra en el hash la variable usuario se le da un valor de `NULL` y se la devuelve.

2_ LOGOUT:

Para el logout lo que se hace es fijarse si la variable usuario tiene valor. Si tiene valor, este valor se vuelve `NULL`. De esta forma el usuario queda deslogueado. Todas estas instrucciones son realizadas en $O(1)$ cumpliendo con los requerimientos del trabajo práctico.

3_ PUBLICAR UN POST:

Para crear este comando se utilizó el hash que contiene los heaps. Para publicar un post se itera todo el hash consiguiendo una complejidad $O(u)$ ya que iterar una estructura basada en arreglos esta es la complejidad de una iteración.

Dentro de cada iteración se crea un post. Lo que tiene una complejidad $O(1)$. Luego al post se le guarda la afinidad del publicador con el usuario del feed. El cálculo de la afinidad se hace restando la distancia entre el publicador y el usuario del feed, y calculando el módulo al resultado de esta resta.

Luego se guarda el post en un heap comparando la afinidad de los post. Se utilizó un heap porque le da prioridad a los elementos según la función comparación que le demos. Además la primitiva del heap de encolar tiene una complejidad de $O(\log(n))$. Los post se guardan en el heap durante cada iteración del hash. de esta forma se guarda el mismo post en cada heap según la afinidad que tenga el publicador con cada usuario.

En cada iteración también se guarda el post en un arreglo. Se guarda según la id del post. Si el post tiene id 2 se guarda en la posición 2 del arreglo. guardar en el arreglo tiene complejidad $O(1)$.

La complejidad total de este comando nos da una complejidad de $O(u \log(p))$.

4_ VER PRÓXIMO POST EN EL FEED:

Para este comando se vuelve a utilizar el hash que contiene los heaps. Primero se obtiene el heap que tiene la clave del usuario que está conectado actualmente y luego se desencola el heap para obtener el post con mayor prioridad. Desencolar en un heap tiene una complejidad $O(\log(n))$ y obtener un dato del hash tiene complejidad $O(1)$.

Luego se obtiene el id del post, el texto, el publicador y la cantidad de likes. Todas estas operaciones tienen una complejidad $O(1)$. Una vez obtenidos estos datos se imprimen en pantalla con el formato adecuado. Luego el post se guarda en una estructura de datos cola para no perder esta memoria y poder liberarla más tarde.

5_ LIKEAR UN POST:

Para el manejo de este comando se decidió usar un árbol binario de búsqueda. Para el abb se usó la función `strcmp` porque nos permite ordenar alfabéticamente el árbol. Para el comando likear, se agrega al árbol el nombre del usuario que likeo, y dicha operación tiene una complejidad $O(\log(u))$, siendo u la cantidad de usuarios. Antes de agregar el usuario al abb, se verifica si este ya había likeado o no. Dicha verificación tiene un costo de $O(\log(u))$. Por ende, para el comando de likear post termina quedando una complejidad total de $O(\log(u))$, que era la pedida en el enunciado.

6_ MOSTRAR LIKES:

Para este comando se aprovechó el abb que contiene quién likeo cada publicación, y se hace un recorrido in-order para poder almacenar una lista de los usuarios que likearon la publicación. Al ser un recorrido in-order, la lista obtenida está ordenada alfabéticamente. Como se van a recorrer todos los nodos del árbol, se tiene una complejidad $O(u)$. Luego, esa lista nos sirve para tener la información y poder hacer lo que queramos con ella. En nuestro caso, imprimimos por pantalla la lista y además mostramos la cantidad de likes que tiene la publicación.